الفصل الصيفي
2023

دفاتر المواد

الأسمبلي

للمهندسة
نوال الزين

Team
Zerone
Computer & Networks Engineers

Assembly language

* مراجعة مادة اللوجيك

D: Decimal O: Octal H: Hexa
B: Binary

25D → B?

8 bit = $2^8 = 256$ → 0 → 256

128	64	32	16	8	4	2	1
0	0	0	1	1	0	0	1

① قارن 25 مع 128
أكبر عوضا 0
أصغر عوضا 1
يساوي ← 1

② عند المقارنة 25 مع 16
كانت 16 أصغر عوضا 0
الآن نطرح 25-16=9
ونقارن الـ 9

9-8 = 1

B → D

128	64	32	16	8	4	2	1
0	0	1	1	1	0	0	1

57D = 32 + 16 + 8 + 1

H → B

3 7 H

8	4	2	1
0	0	1	1
0	1	1	1

→ 0011 0111

Scanned with CamScanner

B → H

8	4	2	1	8	4	2	1
1	0	0	0	1	0	1	1
↓				↓			
8				B			

= 8B H

D = 0, 1, 2 .. 9	10	11	12	13	14	15	16	17
H = 0, 1, 2 .. 9	A	B	C	D	E	F	10	11

Sign and un Sign

* نستخدم فقط bit 16 , 8
لأننا نتعامل مع full capcity for Reg.

8 bit

unSign

	128	64	32	16	8	4	2	1
min = 0 =	0	0	0	0	0	0	0	0
max = 255 =	1	1	1	1	1	1	1	1

$0 \rightarrow FF_H$

8 bit

Sign

	±	64	32	16	8	4	2	1
min = -128 =	1	0	0	0	0	0	0	0
max = +127 =	0	1	1	1	1	1	1	1

1 → ⊟ 0 → ⊞

CS Scanned with CamScanner

Sign

1 1 1 1

(-) ←

↓ 2' complement

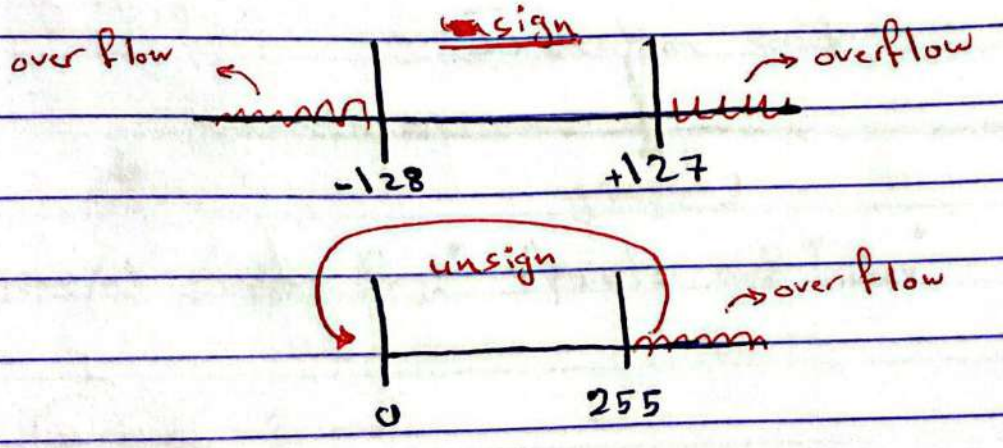
0 0 0 1 → (1) = -1

1 1 1 0 0

(-) ←

↓ 2' complement

0 0 1 0 0 → (4) = -4



255 = 1 1 1 1 1 1 1 1 unsign

1 +

Carry 1 0 0 0 0 0 0 0 0

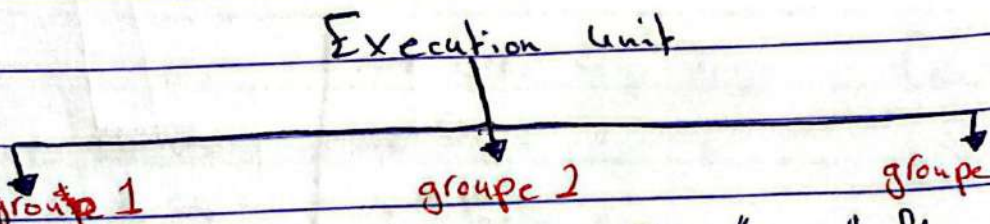
عليه الزيادة على 255 تعود الى الصفر

Veriables → X = 70 → Memory Location

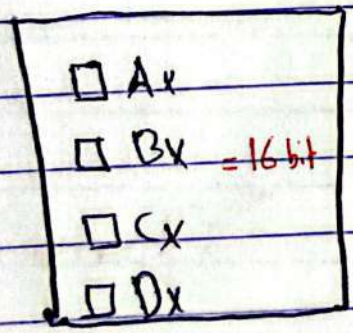
Registers → □ = 70 → موجود داخل cpu أو في واجهة التواصل

* Microprocesser (8086) architecture:

1. Execution unit [EU] وحدة التخزين
2. Bus Interface Unit [BIU] وحدة النقل



Groupe 1 :



Reg size = 16 bit
Word size = 16 bit = 2 byte
Data Bus Reg. = 16 bit
Address size = 20 bit

8 bit	8 bit

عكس تقسيم ال
مزئيين كل جزء
8 bit
(vertical Devision)

AX =	A_H	AL
BX =	B_H	BL
CX =	C_H	CL
DX =	D_H	D_L

Scanned with CamScanner

Bus Interface Unite

① Segement Reg.

② Ip

③ physical address generation

Segement Reg.

CS	→ code seg. Reg.	(للتعليمات)
DS	→ Data " "	(للبيانات)
SS	→ Stack " "	
ES	→ Extra " "	(مساعد للبيانات)

64KB = Seg size حجم

$2^{16} = 64$ KB



"لكل موقع العنوان"

CS Scanned with CamScanner

Groupe 3 : Flag Register

C_F	P_F	OF
S_f	A_f	ZF
D_f	I_F	

$C_F \rightarrow$ $C_F = 1 \leftarrow 1$ ← carry طلع

$C_F = 0 \leftarrow 0$

$C_F = 1$

$$\begin{array}{r} 1101\,0011 \\ 1101\,1011 \\ \hline 1\,1010\,1110 \end{array} \quad +$$

carry 1

$A_f \rightarrow$ First byte تتعامل فقط مع ال

$A_F = 1 \leftarrow 1$ ← 4bit في carry اذا طلع

$AF = 0 \leftarrow 0$

$A_F = 0$

$$\begin{array}{r} {\scriptstyle 1\ 1} \\ 1101\,0011 \\ 1101\,1011 \\ \hline 1\,1101\,1110 \end{array} \quad +$$

carry 1

PF → يتعامل فقط مع ال First byte

يعد عدد ال (1) في الجواب

اذا كان العدد
even → PF = 1
odd → PF = 0

١١ ٥ ١١ ٥ ١١
+ ١١ ٥ ١١ ٥ ١١
١٥ ١ ٥ ٥ ١ ١٥

even → PF = 1 ←

SF → آخر رقم

- = 1
+ = 0

١١ ٥ ١١ ٥ ١١
+ ١١ ٥ ١١ ٥ ١١
١١٥ ٥ ٥ ١ ٥١

SF = 1 ←

ZF → ZF = 1 ← 0 = اذا كان الناتج

ZF = 0 ← 0 ≠ " " "

OF → اذا حدث ⊞ + + ⊞ - او ⊞ - - ⊞ +

يكون OF = 1

عدا ذلك OF = 0

Scanned with CamScanner

Ex: 1111 1111
0000 0001 +
Carry [1] 0000 0000
[1]

$C_F = 1$, $A_F = 1$, $Z_F = 1$, $S_F = 0$
$P_F = 1$ $O_F = 0$

Ex: 4 - 10 calculate Basic flages

Borrow [1] → 0000 0100 (10)
0000 1010 -
1111 1010

$C_F = 1$, $A_F = 1$ $Z_F = 0$, $S_F = 1$
$P_F = 1$ $O_F =$ → 2'compl. هو عملية طرح لكن بالجمع

2' complement → 4 + -10

0000 0100 (4)
1111 0110 + (-10)
1111 1010

$O_F = 0$

CS Scanned with CamScanner

Mov: شرح تعليمة الـ mov

mov [Dest] و [src]

بالسيمبلي يوجد تعليمات لها 0 او 1 او 2 operantes

1 او 8 operantes ←

مبدأ التعليمة: يقوم بنسخ قيمة الـ src على الـ Dest
وهذه التعليمة لا تؤثر على الـ Flages

Dest: Reg. or Memory Location يمكن أن تكون

Src: Reg. or IMM. constant or Memory Location يمكن أن تكون

* يمنع وجود two memory location at the same time

* يمنع وجود different size

* يمنع التعامل مع Sigments مباشرة (Ds, Ss, Es, Cs)

Ds = 3000H ← ممنوع

Scanned with CamScanner

(صحيح) Legal	(غير صحيح) illegal
mov AL , AH mov Ax , Cx	mov AL, Bx mov Bx, AL — different size
mov AL , X mov y , AL	mov y , X → two memory Location at the same T
mov AL , 40H (8bit) mov Ax , 40H → 0040H	mov AL , 466H AL: 8bit — 466H: 12bit different size
mov Ax , 600H mov Ds , Ax	mov Ds , 600H Seg. Register

Comment تعني ; ملاحظة *

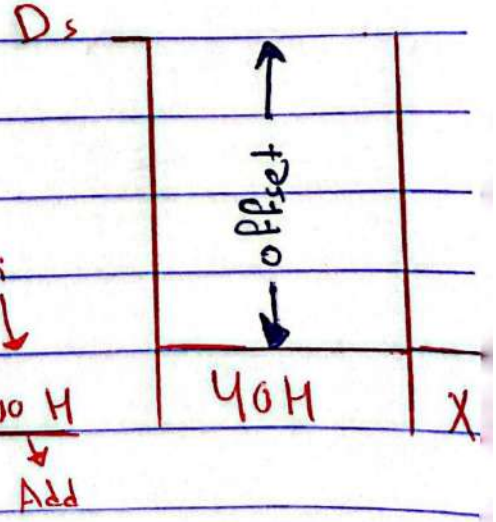
Addresing mode:

(direct)

mov AL , X ; AL=40H

or mov AL , [100H]

[Add.] → محتوى هذا العنوان



Scanned with CamScanner

(indirect)

or
```
mov  si , offset x
mov  AL, [si]
```

Bx , Si , Di , Bp

هذول (فقط) مخصصين لعنوان Address

① Reg. Add. mode
```
mov  AL , AH
mov  Bx , Cx
```

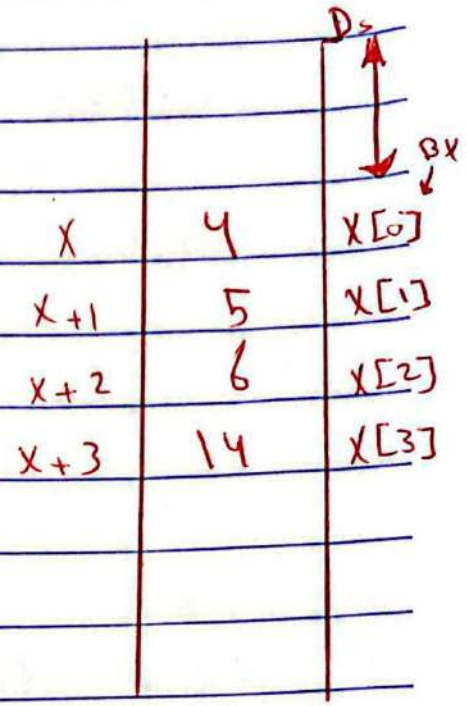
② Direct. Add. mode
```
mov  AL, x
mov  AL, [0100H]
```

③ IMM. Add. mode
```
mov  AL, 40H
```

Scanned with CamScanner

④ Indirect Add. mode

(Si, Di, Bp, Bx) → Pointer




```
mov Bx, offset x
mov AL, [Bx]
```

⑤ Direct index Add. mode

Si Di

```
mov Si, 3
mov X[Si], 14
```

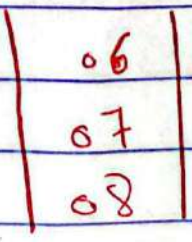
⑥ Base realitive Add. mode

Bx Bp المؤشر على الـ Base نشط ماشي

```
mov AL, [Bx+2] ; AL=6
```

⑦ Base Index Add. mode

Bx Bp Si Di




```
mov Bx, offset x
mov Si, 2
mov AL, [Bx][Si]
```

Scanned with CamScanner

OR Mov AL, [Bx + Si] بنزل لتحت

Mov AL, [Bx - Si] بطلع لفوق

Data — Variables :-

DB →	Define byte	8 bit
DW →	" word (double byte)	16 bit
DD →	" double word	32 bit
DQ →	" 4 words	64 bit
DT →	" Ten byte	80 bit

Ex: طريقة التعريف

X DB 4 ← decimal

04	X

X DB ? → بدون قيمة بخزنه 0

التخزين ←

X DW 2233H

33 H	X
22H	X+1

Scanned with CamScanner

X DW 30H , 16 , 00110011B

00 30H 00 10H 00 33H

30H	X
00	X+1
10H	X+2
00	X+3
33H	X+4
00	X+5

String: طريقة التخزين في

A → 65 → 41H
a → 97 → 61H
0 → 30H
⋮
9 → 39H

→ بايت

X DB 'A' ← one byte

'A'	X

BP إلى

Y DB '7'

41H	X

37H	X

Scanned with CamScanner

X DB 'ABC' or XDB 'A', 'B', 'C'

41	X
42	X+1
43	X+2

'A'	X
'B'	X+1
'C'	X+2

X DW 'ABC'

41	X
null	X+1
42	X+2
null	X+3
43	X+4
null	X+5

EQU : اعطاء قيمة ثابتة لا يمكن تغييرها

X EQU 4

قيمة [X=4] ولا يمكن تغييرها

ويتم التعامل معه مباشرة (غير موجود بالذاكرة)

Scanned with CamScanner

Data transfar instructions:

- [1] Data moving instructions
  Mov , Xchg

[2] Stack inst.
Push , Pop

[3] I/O inst.
In , Out

[4] Address moving instruction
LEA — offset نفس مبدأ عل

[5] Flag moving instructions (AH reg.)
LAHF , SAHF

~~DUP dir.~~

DUP directive: (تكرار)

y DB 3 DUP('A')

'A'
'A'
'A'

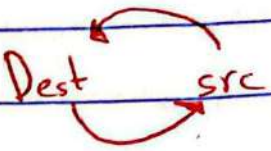
X DB 4 DUP(?)

00
06
00
00

Scanned with CamScanner



X chg: ____ , ____ تبديل القيمة

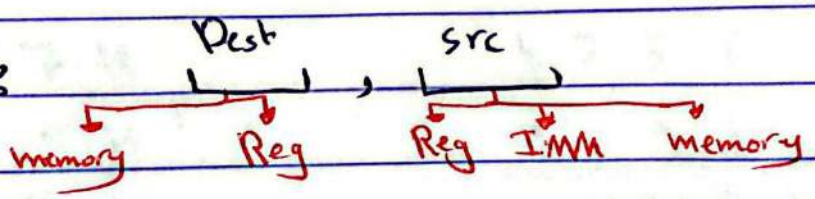



```
mov   AL, 4
mov   BH, 6
Xchg  AL, BH
```

لا يؤثر على ال Flag

AL=6 BH=4

ADD: Dest , src



الإجابة تخزين Dest

جميع ال Flag تؤثر عليها ADD

```
mov   Cx, 3344H
mov   Dx, 1122H
ADD   Dx, Cx
```

```
3344
1122 +
4466 H
0110 0110
```

CX = 3344 H
DX = 4466H

PF = 1 SF = 0
AF = 0 OF = 0
CF = 0 ZF = 0

Scanned with CamScanner

Inc :- Dest ; Dest = Dest + 1

mov CH, 33H
Inc CH
CH = 34H

تؤثر على الـ Flag ما عدا CF

ADC :- Dest (R M), src (R M I) ; Dest = Dest + src + CF

تؤثر على كل الـ Flag

①

37H + 19H = 50H

1237H + 1191H = 2408H

8 , 16 bit لا يوجد مشاكل
لكن اذا كانه أكثر من bit 32

Bx	Ax
3322	7711 H
Dx 2211	Cx 9111 H +
5534	0822

اصناعك
التقسيم

ADD Ax + Cx = 0822
ADD Bx + Dx = 5533

55338	0822
CF = 0	CF = 1

في حالة فضلنا استعمال ADC

ADD Ax + Cx = 0822
ADC Bx + Dx = 5534 + CF

55340822 ✓

Scanned with CamScanner

Ex: تعريف
```
x  DD  33227711H
y  DD  2211911H
z  DD  ?
```

Pointer
```
mov  Si, offset x
mov  Di, offset y
```

```
mov Ax, [Si] ; Ax=7711H
mov Bx, [Si+2] ; Bx=3322H
```

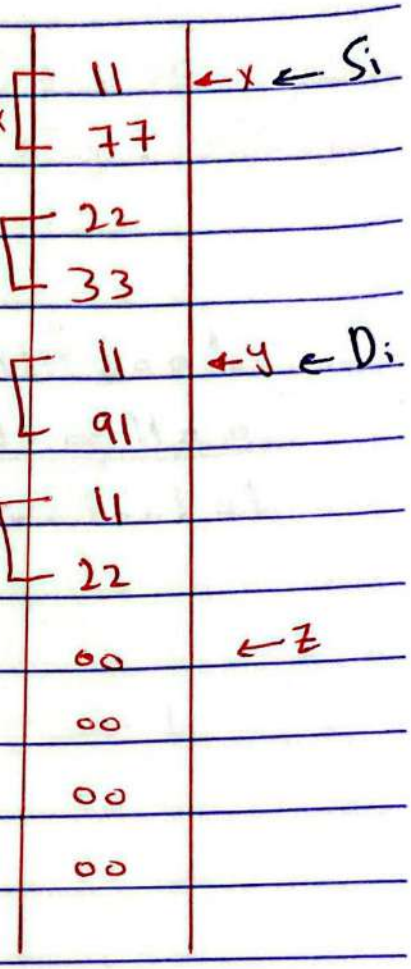
```
mov AL, [Si] → one byte
mov Ax, [Si] → 2 byte
```

```
mov Cx, [Di]  ; Cx=9111H
mov Dx, [Di+2] ; Dx=2211H
```

```
ADD Ax, Cx ; Ax = Ax+Cx = 0822H , CF=1
ADC Bx, Dx ; Bx = Bx+Dx+CF = 5534H, CF=0
```

خلصت من استخدام Si, Di بقدر ارجع استخدمهم

```
mov Si, offset z
mov [Si], Ax
mov [Si+2], Bx
```



Scanned with CamScanner

Sub: Dest , src ; Dest = Dest - src

تؤثر على كل الـ Flag

```
mov CH, 1111 0001 B
mov CL, 1100 1100 B
sub CH, CL
```

1111 0001 -
1100 1100

CL = 1100 1100 B

CH ← 0001 0001

CF = 0 PF = 0 odd

AF = 1 ZF = 0

SF = 0 OF = 0 → 2's complement استخدم

Dec: Dest ; Dest - 1

تؤثر على كل الـ Flag

ما عدا CF

SBB: Dest , src ; Dest = Dest - src - CF

تؤثر على كل الـ Flag

BX AX

3322 6711 H

DX, 2211, CX 9111 H

SBB → 1110 7600 ← Sub

CS Scanned with CamScanner

Ex:

```
X  DD  33220711 H
Y  DD  2211 9111 H
Z  DD  ?

mov Si, offset x
mov Di, offset y

mov Ax, [Si] ; Ax = 0711 H
mov Bx, [Si+2] ; Bx = 3322 H
mov Cx, [Di] ; Cx = 9111 H
mov Dx, [Di+2] ; Dx = 2211 H

SUB Ax, Cx ; 7600 H , CF = 1
SBB Bx, Dx ; Bx = 1110 , CF = 0

mov Si, offset z
mov [Si], Ax
mov [Si+2], Bx
```

11	← x ← Si
07	
22	
33	
11	← y ← Di
91	
11	
22	
00	← z
00	
00	
00	

CS Scanned with CamScanner

Ex: mov DH, 255
INC DH

1111 1111
+ 1
[11] 0000 0000

SF=0 OF=0
ZF=0 PF=1
AF=1
CF → [11] المفروض يكون الجواب

لكن انتبه INC لا تؤثر على CF
فبتفضل زي ما كانت عليه. CF=0

CS Scanned with CamScanner

17/30

Assembly

Cmp: Dest , src ; Dest - src (مقارنة)

* يؤثر على كل ال Flag

* لا يغير قيمة ال src او Dest

الهدف منه التأثير على ال Flag للمقارنة

Control Transfar instruction

- تعليمات غير مشروطة → Unconditional
- Conditional

Unconditional:
- JMP
- Call
- RET

Conditional:
- Relatinal
  - Signed
    - JG
    - JNG
    - JGE
    - JL
    - JNL
    - JLE
    - JE
    - JNE
  - unsigned
    - JA
    - JNA
    - JAE
    - JB
    - JNB
    - JBE
    - JE
    - JNE
- Flag Ins.
  - JZ
  - JNZ
  - JC
  - JNC
  - JO
  - JNO
  - JP
  - JNP
  - JS
  - JNS

Scanned with CamScanner

17/30

Assembly

Flag Ins.

Z
- JZ: Jump if zero, ZF = 1
- JNZ: " " not zero, ZF = 0

C
- JC: Jump if carry, CF = 1
- JNC: " " not carry, CF = 0

O
- JO: Jump if overflow, OF = 1
- JNO: " " not " , OF = 0

P
- JP: Jump if parity even, PF = 1
- JNP: " " not " " , PF = 0

S
- JS: Jump if sign, SF = 1
- JNS: " " not " , SF = 0

Scanned with CamScanner

17/30

Assembly

Signed

- G
  - JG: Jump if greater, Dest > src
  - JNG: " " not ", Dest ≤ src
  - JGE: " " greater, Dest ≥ src

- L
  - JL: Jump if Less, Dest < src
  - JNL: " " not ", Dest ≥ src
  - JLE: " " Less or equal, Dest ≤ src

unsigned

- A
  - JA: Jump if above, Dest > src
  - JNA: " " not ", Dest ≤ src
  - JAE: " " above or equal, Dest ≥ src

- B
  - JB: Jump if below, Dest < src
  - JNB: " " not ", Dest ≥ src
  - JBE: " " Below or equal, Dest ≤ src

Signed and unsigned

- E
  - JE: Jump if equal, Dest = src
  - JNE: " " not ", Dest ≠ src

CS Scanned with CamScanner

The case in general:

Jump Label
Ins.

Ex: JZ L_1 , JMP L_2

Ex:
```
Mov AL, 4
Mov BL, 6
JMP L2          → ينتقل الى L2
X ADD AL, 6     → لا ينفذ التعليمة
L2: ADD AL, 2
```
AL = 6 BL = 6

FFH
- → JA (unsigned) = 255
- → JG (signed) = -1

Scanned with CamScanner

Ex8 mov AL, ØFDH ; AL=-3
8bit

cmp AL, -1
JA L1
JNC AL ; AL=-2
Jmp L2

L1: Dec AL
L2: HLT

اذا كبر يكبر واصر
Jump if AL greater than -1

JG تتعامل مع sign ال
Signed ← FDH
1111 1101 ← سالب
-3 : 0000 0011

Ex: AL, 0FDH ; AL=253

cmp AL, -1
255

JA L1

Inc AL ; AL=254

Jmp L2

L1: Dec AL

L2: HLT

AL = 254 = FEH

unsign ← JA

253 = FDH = 1111 1101

-1 = 0000 0001

255 = 1111 1111

In assembly:-

org 100H

HLT

instruction code

Data

Scanned with CamScanner

In C++

```
if (Avg >= 50)
  R = "P"
else
  R = "F"
```

In assembly

```
ORG 100H
cmp Avg, 50
JAE L1
mov R, 'F'
Jmp L2
L1: mov R, 'P'

L2: HLT

Avg DB 70
R DB ?
```

Instruction code

Data

R = 'P'

In C++

```
i = 3    X = 0
while (i >= 0)
{ i--;
  X += 3;
}
```

In assembly

```
ORG 100H
mov cl, 3
L2: cmp cl, 0
JL L1
Dec cl
ADD X, 3
Jmp L2
L1: HLT

X DB 0
```

Scanned with CamScanner

Ex: X ← array عبارة عن

X DB 1, 2, 3, 4, 5

01	X
02	X+1
03	X+2
04	X+3
05	X+4

Solution 1

```
mov AL, X
ADD AL, X+1
ADD AL, X+2
ADD AL, X+3
ADD AL, X+4

mov sum, AL
```

(طريقة غير عملية)

Solution 2

```
mov Si, offset x
mov Cx, 5
mov AL, 0
L1: mov AL, [Si] ; AL=01
ADD sum, AL ; sum=01
INC Si
Dec cx
cmp cx, 0
JNE L1
```

or

```
Dec cx
JNE L1
```

Cx=0 .. Cx=1 .. Cx=2 .. Cx=3 .. Cx=4 .. Cx=5

ZF=1 ZF=0 ZF=0 ZF=0 ZF=0 ZF=0

↳ ZF=1 للتعويض قيمة ما بدل قفز

* JZ = JE
JNE = JNZ

Scanned with CamScanner

Ex: استبدل عناصر المصفوفة السالبة بصفر

Arr DB -1, 2, 3, -4, -5, 7

-1
2
3
-4
-5
7

Solution 1

```
mov Bx,0
mov cx,6
L2: cmp Arr[Bx],0
    JG L1
    mov Arr[Bx],0
L1: INC Bx
    Dec cx
    JNZ L1
```

Solution 2

```
mov Bx,0
L2: cmp Arr[Bx],0
    JG L1
    mov Arr[Bx],0
L1: INC Bx
    cmp Bx,5
    JBE L2
```

* mull: operater

R M

8 bit → Ax = AL * op
(ممكن يطلع الناتج أكبر من 8 bit فيخزن بـ 16 bit)

16 bit → Dx.Ax = Ax * op
(ممكن يطلع الناتج أكبر من 16 فيخزن بـ 16)

Scanned with CamScanner

Ex: X DB 4
y DB 2
mov AL, X ; AL=4
mull y ; Ax = AL * y = 0008 H

AH=00H AL=08H

Ex: X DW 4
y DW 3
mov AL, X ; AL=4
mull y ; Dx.Ax = Ax * y = 000C H

Dx= 0000H Ax= 000CH

* Div: operater (unsigned)

R M

8bit → Ax = Ax / op

16bit → Dx.Ax = Dx.Ax / op

Dx: الباقي Ax: الناتج

AH | AL → النتيجة

AH: الباقي

Scanned with CamScanner

Ex: mov Ax, 7
mov Bl, 3
Div Bl
8 bit

→ Ah, 0
Al, 7

AL = 02H AH = 01H

Ax = 0102H

Ex: mov Dx, 0
mov Ax, 7
mov Bx, 3
Div Bx ; Dx, Ax = Dx, Ax | Bx = 02
16 bit
01H 02H

* Imull , I Div (signed)

mov Al, -4
mov Bl, 2
Imull Bl ; Ax = Al * Bl = -8
8 bit

-4 2

0000 0000 0000 1000 = 8

Ax = 0FFF8H = 1111 1111 1111 1000

Scanned with CamScanner

0000 0100 = 4

Ex: mov AL, -4 ; 1111 1100 = 252 = AL

mov BL, 2 252 2

mul BL ; AX = AL * BL = 504 D

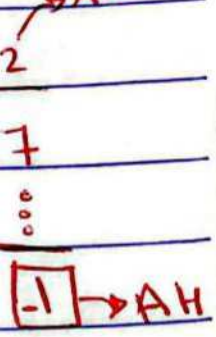
unsigned 8bit

Ex: mov AX, -7

mov BL, 3

IDiv BL ; AX = AX / BL

signed 8bit



AH = -1 ← (AX) إشارة الباقي هي نفس إشارة المقسم

يكون signed فقط

* لنفترض أنه كتبت الكود هكذا:

mov AL, -7 → AX = 00 -7

mov BL, 3 ↓ موجب

وهاي بتتحول 249 لأنه

صار الرقم موجب

Scanned with CamScanner

Ex: استبدل الارقام الزوجية بصفر

```
X DB 2,1,5,4,6
mov Si,offset x
mov cx,5
mov BL,2
L2: mov AH,0
    mov AL,[Si]
    Div BL
    cmp AH,0
    JNE L1
    mov [Si],0
L1: INC Si
    Dec cx
    JNZ L2
```

Si → X

02
01
05
04
06

AH= الباقي AL= الناتج

* mov Ax,X → error
  mov Ax,[Si] → 0102H
  1 byte في 2 byte يحجز

Scanned with CamScanner

```
X DW 2,1,5,4,6
   mov Ax,[si] ✓
Ax = 0002 ✓
```

02
00
01
00

```
Ex: X DW 2,1,5,4,6
   mov Si,offset X
   mov cx,5
   mov BL,2
L2: mov Ax,[si] ; Ax=0002H
   Div BL
   Cmp AH,0
   JNE L1
   mov w.[si],0
L1: ADD si,2
   Dec cx
   JNZ L2
```

Scanned with CamScanner

Ex: 5! احسب ناتج

```
mov AH, 0
mov AL, 1
mov BL, 5
L1: mul BL      ; AX = AL * BL
    Dec BL
    JNZ L1
```

AX = 120D , BL = 00H

* ~~if Loop not Label~~

* Loop : Label
  - ① Dec CX
  - ② Jump to Label if CX ≠ 0 (JNZ)

* LoopE Label
  - ① Dec CX
  - ② Jump to Label if
    - CX ≠ 0
    - &
    - ZF = 1

Scanned with CamScanner

* Loop NE : Label

- ① Dec cx
- Jump to Label if
  - cx ≠ 0
  - &
  - ZF = 0

Scanned with CamScanner

* Loop NE : Label

① Dec cx

Jump to Label if → cx ≠ 0 & ZF = 0

* JCXZ : Jump if CX = Zero

JCXZ Label

Ex:
```
Mov CL, [100H]
Mov CH, 0
JCXZ Exit
Mov AH, 2
Mov DL, '*'
L1: INT 21H
Loop L1
Exit
```

Scanned with CamScanner

NOTE for Loop:

if cx reg. contained zero, any Loop you create using the Loop instruction will repeat 65,536 times

Ex: Mov Ax,0
Mov Cx,5
L1: ADD Ax,100
CMP AH,0
LoopE L1

توضيح:

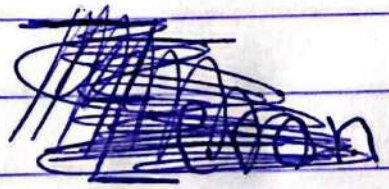
#Loop	Ax	AH	Cx	ZF (cmp AH)
0	0	0	5	0
1	0064 H	00	4	1
2	00C8 H	00	3	1
3	012C H	01	2	0 >> Exit Loop

CS Scanned with CamScanner

Online

Ex: X DB 9,8,7,16,5

```
Mov Si, 0
Mov Cx, 5
L1:
    Mov AL, X[Si]
    INC Si
    Cmp AL, 7
    Loop NE L1
```



^^

ZerOne

team

Scanned with CamScanner

Bit Manipulation

1 SHR : "shifting to Right"



يخزن في الـ carry flag

SHR Dest , shifting value

Dest: R, M

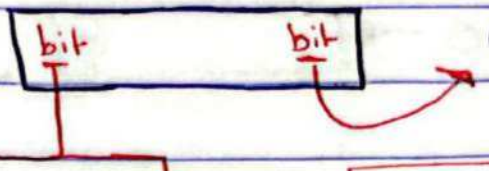
shifting value: CL, immadiate = 1

Ex:
```
Mov AL,01100001b
SHR AL,01 ;AL=00110000 b
Mov CL,03
SHR AL,CL ;AL=00000110 b
```

* Shifting 03 → يعني إدخال 3 أصفار

Scanned with CamScanner

[2] SAR: "Arithmetic Shifting to Right"



اذا كانت قيمة ال bit = 0 يتم ادخال 0

اذا كانت قيمة ال bit ← 1 يتم ادخال 1

* الاختلاف بين SHR , SAR

SAR ← uses the sign flag bit to fill

" الرقم اذا كان سالب يظل سالب
واذا كان موجب يظل موجب "

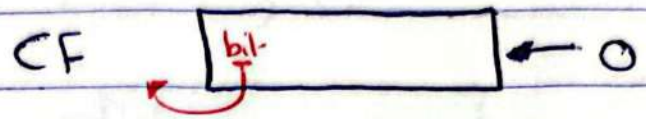
SAR Dest , Shifting value

Ex:
```
Mov  BL, 10110111b ;BL= -73
SAR  BL, 01 ; BL=11011011 = -37
Mov  cL, 02
SAR  BL, cL ; BL = 1111 0110b = -10
```

Scanned with CamScanner

18/6 online

[3] SHL: "Shifting to Left"



SHL dest , shifting value

Ex:

```
Mov DL, 01100001b
SHL DL, 01 ; DL= 11000010 b
Mov CL, 03
SHL DL, CL ; DL= 00010000 b
```

[4] SAL: "arithmetic shifting Left"

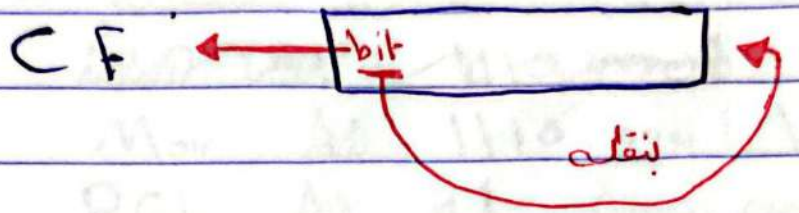
* نفس عمل SHL تمامًا

ZeirOne ^^
team

Scanned with CamScanner

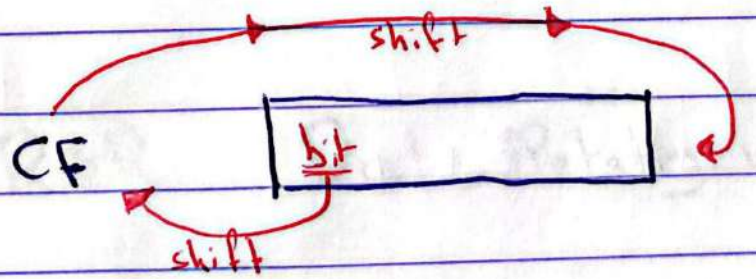
5 ROL: "Rotating to left"

ROL (Dest , Rotating value)



Ex:
```
Mov AL, 11100001 b
ROL AL, 01 ; 11000011 b
Mov CL, 03
ROL AL, CL ; 00011110 b
```

6 RCL: "Rotating Carry Left"



القيمة الموجودة في الـ CF بنقلها لأقصى اليمين
وآخر bit موجود بنقله لـ CF

RCL (Dest , Rotating value)

Scanned with CamScanner

Example:

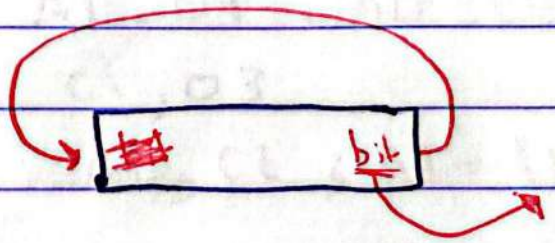
* طبعاً هو لازم يكون في قيمة لـ CF او عادتين شو قيمته

* assume initial CF = 1

~~Mov AL, 1110 0001~~

```
Mov  AL, 1110 0001 b
RCL  AL, 01 ; AL= 11 000 11 b
Mov  CL, 03
RCL  AL, CL ; AL = 000 111 11 b
```

7 ROR: "Rotating to Right"

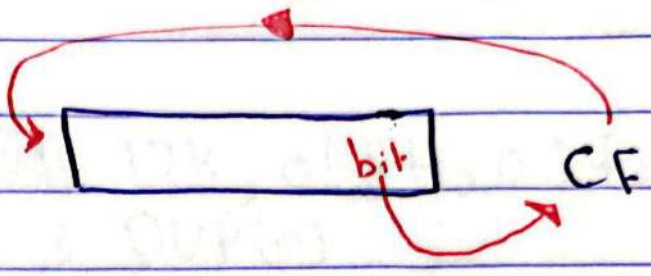


ROR Dest, Rotating value

```
Ex: Mov  AL, 1110 0001 b
    ROR  AL, 01 ; AL= 1111 0000 b
    Mov  CL, 03
    ROR  AL, CL ; AL= 000 1111 0
```

Scanned with CamScanner

⑧ RCR : "Rotating cary Right"



RCR Dest, Rotating value

Ex: assume initial CF = 1

```
Mov AL, 111 00001
RCR AL, 01 ; AL= 1111 0000
Mov cl, 03
RCR AL, CL ; AL= 111 11110
```

ZerOne ^^

team

Scanned with CamScanner

Ex: كم عدد الـ [1] الموجودة في المصفوفة ثم يتخزين القيمة في المصفوفة Y

```
X DB 12H, 055H, 0A7H
Y DB 3 DUP(0)

ORG 100H
Mov Si, 0
L3: Mov cl, 0
Mov AL, X[Si]

L2: SHL AL, 1
    JNC L1
    INC cL
L1:
    CMP AL, 0
    JNZ L2
    Mov Y[Si], cl
    INC Si
    Cmp Si, 2
    JBL L3
    HLT
```

Si

0	1	2
12	055	0A7H

00010010 ← 00100100 ;

Scanned with CamScanner

Ex: استبدال الارقام الزوجية بصفر بدون استخدام IDIV or DIV

```
X DB 10, 3, 1, 2
ORG 100H
Mov Si, offset x
Mov Cx, 4
L1: Mov AL, [Si]
SHR AL, 1
JC L2 ; ODD
Mov [Si], 0 ; Even
L2:
INC Si
Loop L1
HLT
```

4 = 100
6 = 110
3 = 011
5 = 101

* اذا في Carry ← odd
* اذا فش Carry ← even

Ex: احسب عدد الأرقام الزوجية والفردية في العناصر

```
X DB 10, 3, 1, 2

Even DB ?
ODD DB ?
Mov Si, 0
Mov Cx, 4
L3: Mov AL, X[Si]
SHR AL, 1
JC L1
JNC even
Jmp L2
L1: INC odd
L2: INC Si
Loop L3
```

Scanned with CamScanner

18/8 Online

Logical instructions

[1] AND:

AND Dest, src

Dest: R, M

src: R, M, I

R, M → can be 8 or 16 bit

Ex:

```
Mov AL, 0110 0001b
AND AL, 1101 1111b ; AL=0100 0001b
```

مراجعة وتذكير:-

num.	&	num.	Result
1	&	1	1
1	&	0	0
0	&	1	0
0	&	0	0

[2] OR:

OR Dest, src

Ex: Mov AL, 0100 0001b
OR AL, 0010 0000b
; AL=0110 0001b

num.	OR	num	Result
1	OR	1	1
1	OR	0	1
0	OR	1	1
0	OR	0	0

CS Scanned with CamScanner

[3] NOT: NOT Dest

num	not	Result
1	not	0
0	not	1

Ex: Mov AL, 00011011
NOT AL ; AL=11100100

[4] NEG: "2' complement" (تعكس الإشارة)

NEG Dest ; Dest = 0 - Dest

Ex: Mov AL, 5 ; AL= 05H ⇝ 00000101
NEG AL ; AL = 0FBH (-5) ⇝ 11111011
NEG AL ; AL = 05H ⇝ 00000101

Mov AL, 5
NOT AL ; 1' complement هاي بتطلع
ADD AL, 1 ; 2' compl. هاي بترجع
} Equivalent for NEG

Scanned with CamScanner

5 XOR:

XOR Dest, src

Num	XOR	Num	Result
1	XOR	1	0
1	XOR	0	1
0	XOR	1	1
0	XOR	0	0

Ex: Mov AL, 00000111b
XOR AL, 00000010b
; AL = 00000101b

6 TEST: TEST Dest, src

* بتشتغل نفس شغل ال AND لكن الفرق انو ما بتخزن الناتج

الهدف منه انها تأثر على ال Flag {SF, ZF, PF}

Ex: Mov AL, 00000101
TEST AL, 1 ; ZF = 0
TEST AL, 10b ; ZF = 1

TEST AL, 10b
AL = 00000101
01b = 00000010
Result = 00000000
ZF = 1

TEST AL, 1
AL = 00000101
1 = 00000001
Result = 00000001
ZF = 0

Scanned with CamScanner

Ex: Convert capital letters to small letters

STR DB 'HELLO'

bit 0

0100 0001 ← A → 41 H

0110 0001 ← a → 61 H

لاحظ أن الإختلاف في الـ bit رقم 5

```
      0100 0001   A
OR→   0010 0000   = 20H
      ---------
      0110 0001   a
```

```
ORG 100H
Mov Bx, offset STR
Mov Cx, 5
L1: OR [Bx], 20H
INC Bx
Loop L1
HLT
STR DB 'HELLO'
```

Scanned with CamScanner

1818 Online

Ex: افحص اذا كان X عدد موجب او سالب بدون استخدام DIV , IDIV
اذا كان موجب خزن 'E' في BL
" " سالب " '0' في BL

X DB 4 →

Solution 1

```
ORG 100H
Mov AL, X
AND AL, 01H
JNZ L1
Mov BL, 'E'
L1: Mov BL, '0'
   END:
   HLT
   X DB 4
```

Solution 2

```
ORG 100H
TEST X, 01H
JNZ L1
Mov BL, 'E'
JMP END
L1: Mov BL, '0'
END:
HLT
X DB 4
```

الارقام الفردية اول bit يكون 1 ← 3 = 011
" " الزوجية " " 0 ← 2 = 010

ZF=0 = 001 ← 7 = 111 AND 001
ZF=1 = 000 ← 2 = 010 AND 001

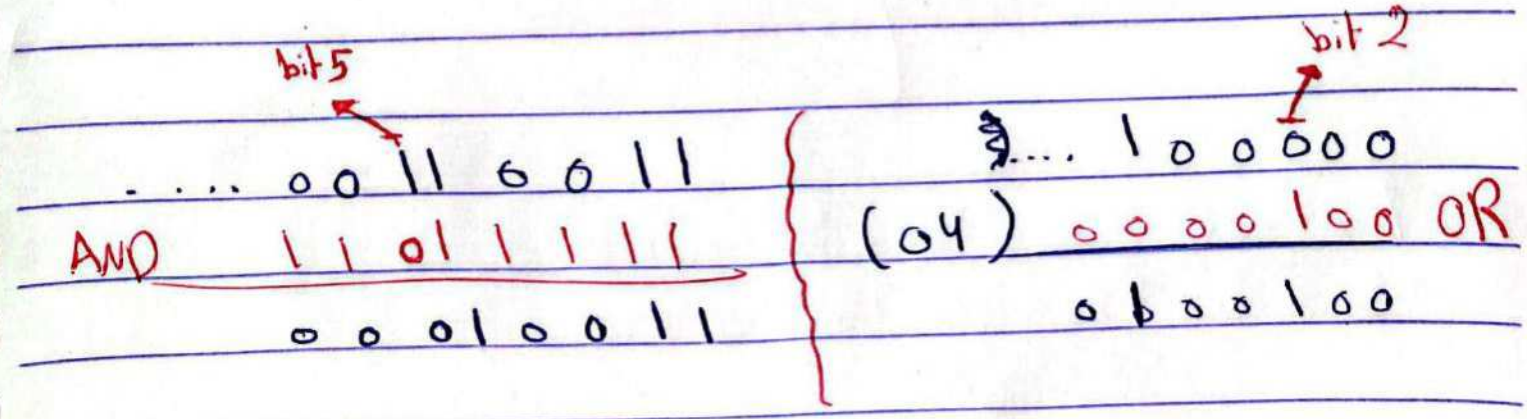
Scanned with CamScanner

Ex: استبدل ال bit رقم 2 بـ 1 في المتغير Y
Z " " 0 " 5 " " " "

```
Y DW 20H, 43H, 51H
Z DW 33H, 62H, 74

ORG 100H
Mov Di, 0
Mov Bx, 3
L1: OR Y[Di], 04H
    AND Z[Di], 0DFH
    ADD Di, 2
    DEC Bx
    JNZ L1
    HLT
```

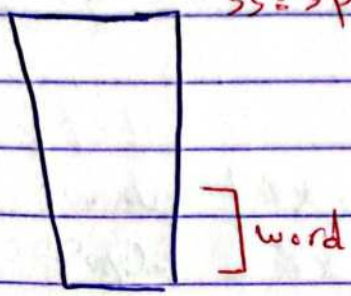
Set → OR
Reset → AND



Scanned with CamScanner

STAck

* جزء من ال memory يتم التعامل معها لتخزين البيانات بشكل مؤقت



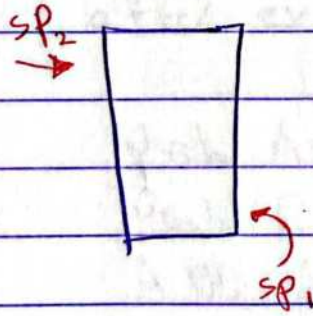
* يخزن كل word
* ويزاح كل word

SP → stack pointer

1. Push op

Reg M Imm

just 16 bit

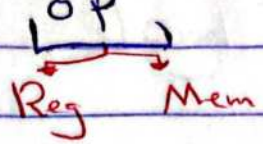


تستخدم لتخزين البيانات في ال stack

1. $SP_1 = SP_2 - 2$
2. تخزن

Scanned with CamScanner

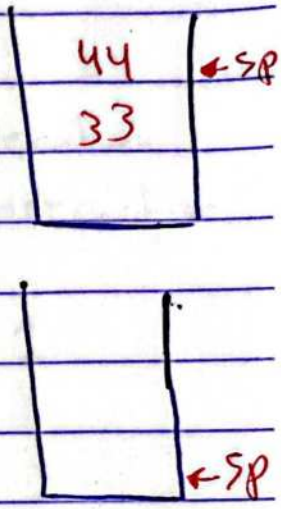
2 POP



1 استرجاع

2 SP = SP + 2 * تستخدم مع الـ stack

Ex: mov AX, 3344H
Push AX
POP BX ; 3344H



Ex: if SP = F11F H, what is value after executing the following program ?

Push AX ; SP = F11
Push BX
ADD AX, BX
POP AX
push CX

Scanned with CamScanner

18/16

String Instructions

REP: تعمل تكرار لتنفيذ تعليمة الـ string طالما $CX \neq 0$

REPZ / REPE:

while → $CX \neq 0$
while → $ZF = 1$
→ [1] repeat execution of string instruction
[2] $CX = CX - 1$

REPNZ / REPNE:

while → $CX \neq 0$
while → $ZF = 0$
→ [1] repeat execution of string instruction
[2] $CX = CX - 1$

Scanned with CamScanner

Strings تتعامل مع الحروف

```
x DB 'Hello'
y DB 5 DUP(' ')
mov Si, offset x
mov Di, offset y
mov cx, 5
L1: mov AL, [Si]
    mov [Di], AL
    Inc Si
    Inc Di
    Loop L1
```

يمكن استبدال هذه التعليمة واحدة من تعليمات الـ string MOVSB

$D_F = 0$ → Left to Right

$D_F = 1$ → Right to Left

1. MOVSB → MOVSW → MOVSD
   - ① copy one byte from [Si] to [Di]
   - ② D_F → 0 → Inc Si , Inc Di
     D_F → 1 → Dec Si , Dec Di

Scanned with CamScanner

Right to Left ($D_F = 1$)

```
X DB 'Hello'
Y DB 5 DUP('')
mov Si, offset X+4
mov Di, offset y+4
   mov Cx, 5
    STD
   L18 movSB
    Loop L1
```

STD → $D_F = 1$
CLD → $D_F = 0$

① MOVSW: Copy word from [Si] to [Di]

② D_F → 0 → ADD Si, 2 , ADD Di, 2
D_F → 1 → Sub Si, 2 , Sub Di, 2

Scanned with CamScanner

[2] LODSW: copy word from [Si] to Ax
LODSB: ① " Byte " [Si] to AL

② DF → 0 → Inc Si
→ 1 → Dec Si

[3] STOSW: copy word from Ax to [Di]
STOSB: ① " Byte " AL to [Di]

② DF → 0 → Inc Di
→ 1 → Dec Di

Ex:
```
X DB 'Hello'
y DB 5 Dup(?)
mov Si, offset X
mov Di, offset y
mov ECX, 5
CLD ; DF=0
L1: LODSB ; Si → AL , AL='A'
    STOSB
    Loop L1
```

Scanned with CamScanner

[4] SCASB: ① compare AL, [Di]
SCASW: " Ax, [Di]

* effect of ZF.

② DF → 0 → Inc Di
→ 1 → Dec Di

Ex: هل الحرف e موجود ؟

```
X DB 'abcdefgh'
mov AL, 'e'
. mov Di, offset x
mov cx, 8
SLD ; DF=0 , L to R
mov cx, 8
L1: SCASB
LoopNE L1
JNZ L2
mov BL, 'F'
Jmp L3
L2: mov BL, 'N'
L3: HLT
```

Scanned with CamScanner

5 CmpSB: ① cmp byte [Si], [Di]
CmpSW: " word [Si], [Di]

② DF → 0 → Inc Si, Inc Di
→ 1 → Dec Si, Dec Di

Ex8 X DB 'Ahmad'
Y DB 'AhmaD'

تتأكد من الجملتين
اذا كانوا متشابهتين

```
LEA Si, X
LEA Di, Y
mov cx, 5
REPE cmpSB
    JZ L1
    mov BL, 'NE' ; not Equal
    Jmp L2
L1: mov BL, 'E'
L2: HLT
```

Scanned with CamScanner

Interrupts

Int : 0 , 1, ... , 255
- 0 → 256 Func
- 255 → 256 Fan

* Int 10H
  Int 21H
  يعنا فقط هذول

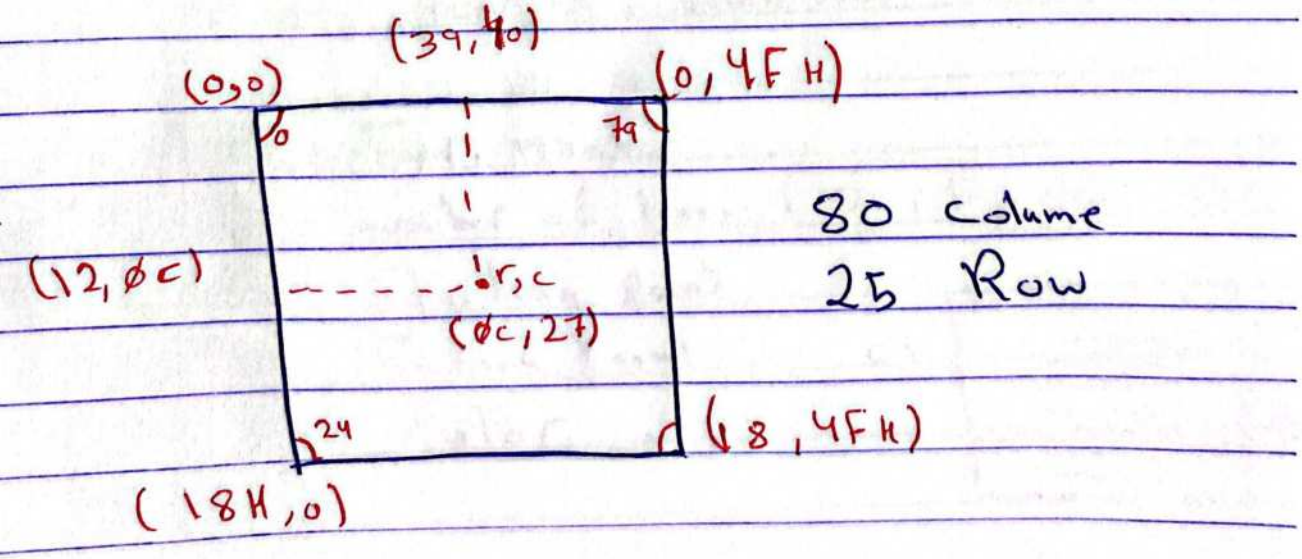
mov AH → Fun. يختار

...
Int — H

	CS	W1
منا وينا ابدأ من؟ ←	IP	W2
offset في الذاكرة؟		W3
		W4

وينا موجود الكود

1) Int 10 H
Function 02 H ; set cursor الكرسر



Scanned with CamScanner

What you need to use this Int ?

1. AH : Fun. number
2. DL : colume number
3. DH : Row number
4. BH : page number (0 → 7 pages)

Ex: Set the cursor in the middle screen

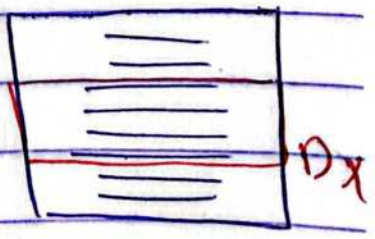
```
mov AH, 02H
mov DH, 0cH ; Row
mov DL, 27H ; colume
mov BH, 0
Int 10H
```

2. Int 10H

Function 06H / 07H ; clear screen

06H: scroll up
07H: scroll Down

1. AH : Fun. number
2. AL : number of lines (0 if clear all)
3. Cx : Starting point
4. Dx : Ending point
5. BH : color



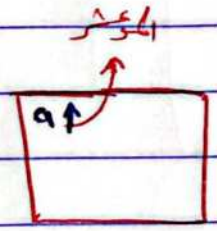
Scanned with CamScanner

Ex. for color 7 1 H
Background Fonet

0 → Black , 1 → white

3 Int 21 H
Function 02 H, print (one byte) on screen

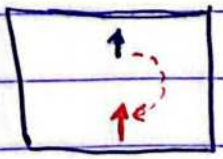
1 DL: byte to print
2 AH: Func. number



Ex: mov DL, 'a'
mov AH, 02 H
Int 21 H

Note: DL = 0AH → new Line (\n in C++)

DL = 0DH → carrege return



Scanned with CamScanner

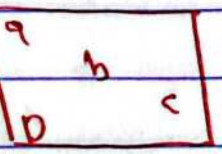
* Int 21 H طباعة كاملة //

Function 09 H ; prent stream on screen

Note: The msg end by $

1. Dx: offset of msg
2. AH: Function number

Ex: MSg DB ' welcome to jordan $'
mov AH, 09H
mov Dx, offset msg
Int 21 H

Ex: طباعة هاي بالشكل ← 

MSg DB 'a', 0AH, 'b', 0AH, 'c', 0AH, 'D', '$'

mov AH, 09 H
mov Dx, offset msg
Int 21 H

Scanned with CamScanner

Int 21 H

(Cin>> in C++)

Function 01 H ; input one byte from keyboard

Note: Store enterd byte in AL

Ex: mov AH, 01 H = Cin >> AL
Int 21 H

* Input and output always in asci code
AL = '1' → AL = 31 H

Ex: Enter 2 number and add them → print them

```
mov AH, 01 H
Int 21 H    ; √2   AL='2' = 32 H
mov BL, AL         BL= 32 H
Int 21 H    ; √4   AL = '4' = 34 H
AND AL, 0F H ; AL=04
AND BL, 0F H ; AL=02
ADD AL, BL   ; AL=06
OR AL, 30H
Mov AH, 02 H
mov DL, AL
Int 21 H
```

√6

32 and
0F
02

06 or
30
36 H

Scanned with CamScanner

Int 21 H

Function 0AH ; Input stream of byte from Keyboard

1. AH : Function number
2. Dx : offset parameter List

Parameter List DB max, ?, Data

actual, msg

PL 20, ?, Data → 20 Dup(' ')

19 → حرف فقط

Ex: PL DB 20H, ?, 20 Dup(' ')

```
mov AH, 0AH
mov Dx, offset PL
 Int 21H
mov PL[18], '$'
mov AH, 09H
mov Dx, offset PL+2
 Int 21H
```

my name is: Ahmad

20	PL0
16	PL+1
m	PL+2
y	PL+3
␣	
n	
a	
m	

عدد الأحرف

Scanned with CamScanner

Procedures

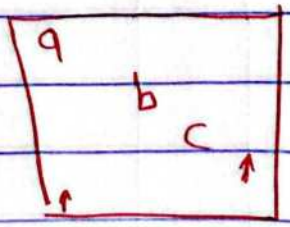
Proc:

```
ORG 100H
ooo
Ip → Call name
ooo
HLT
name PROC
ooo
RET
ENDP
```

Ex:

```
ORG 100H
mov DL, 'a'
call print
mov DL, 0AH
call print
mov DL, 'b'
call print
HLT
print PROC
mov AH, 02H
Int 21H
RET
print ENDP
```

```
mov DL, 0AH
call print
mov DL, 'c'
call print
```



Scanned with CamScanner

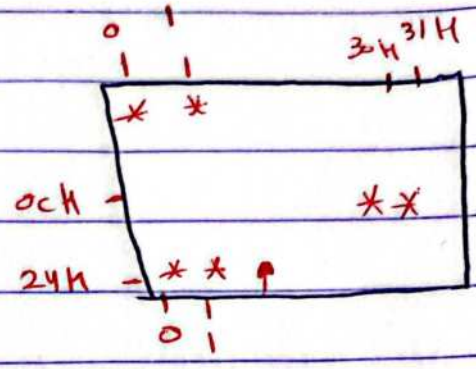
Ex:
```
ORG 100H
mov cx, 8
mov Di, offset x
L1: mov DL, [Di]
    Call print
    Inc Di
    Loop L1
    HLT
X DB 'a', 0AH, 'b', 0AH, 'c', 0AH, 0DH, 'D'
print proc
    mov AH, 02H
    Int 21H
    RET
    Print endP
```

ZerOne team

Scanned with CamScanner

Ex: ORG 100H

```
call print
mov DL, 30H
mov DH, 0CH
call set c
call print
mov DL, 0
call set
mov DH, 24H
call set c
call print
 HLT
Msg DB '**$'
print proc
mov  AH, 09H
mov Dx, offset msg
Int 21H
 RET
print endp
Set c proc
mov AH, 02H
mov BH, 0
Int 10H
 RET
print, Set c, endp
```



Scanned with CamScanner

Parallel port interfacing

Type of parallel port: (SPP, ECP, EPP, ECP+EPP)

* يعالج مجموعة من الـ bits في نفس الوقت ← in parallel

SPP (STanderd Parallel Port)

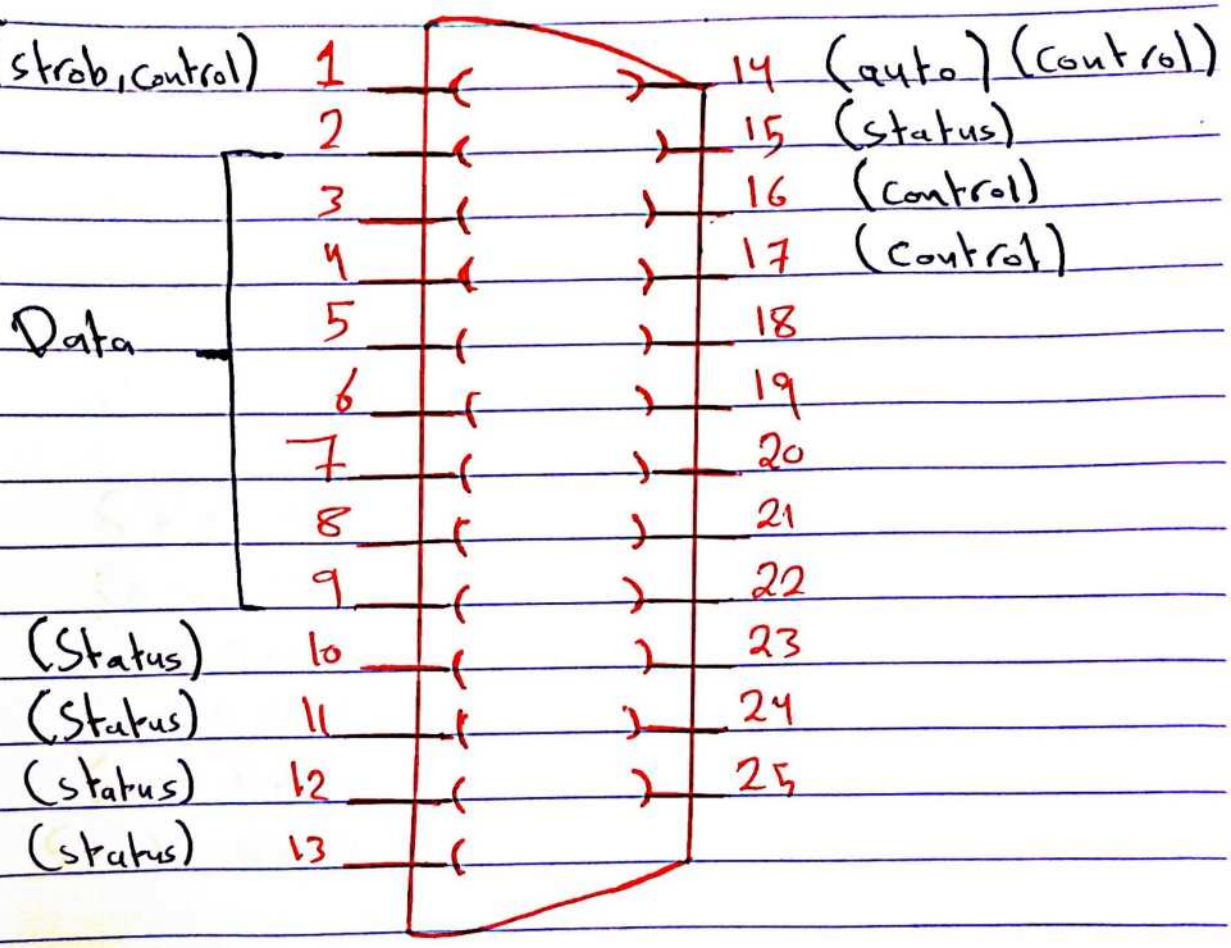
1. has 8 Data lines
2. has 5 Status lines
3. has 4 control lines

In total has 12 output, 5 input

Port	Direction	ADDress	# of used bits	Used Bits
Data	output	378H	8	D0 - D7
statue	input	379H	5	S3 - S7
control	output	37aH	4	C0 - C3

Scanned with CamScanner

* Parallel port (25) Pin



Scanned with CamScanner

* Data port (378H)

2, 3, 4, 5, 6, 7, 8, 9

→ $D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7$

* Status port (379H)

(0) (1) (1) (0) (1) (1) (1) (1)

$S7', S6, S5, S4, S3, S2, S1, S0$

→ 11', 10, 12, 13, 15

S7': Busy (0)
S6: acknowledge
S5: paper out
S4: Select in (printer online or not)
S3: Error
S2 to S0: reserved (محجوزة)

if ~~the~~ prenter is connected
the value = 87 H

1 0 0 0 0 1 1 1

Scanned with CamScanner

* Control Port (37AH)

$C_7, C_6, C_5, C_4, C_3, C_2, C_1, C_0$

— — — — 17 16 , 14 , 1

C_7 - C_5 & reserved

C_4: Interrupts (0 & Disabled)

C_3: Always 1

C_2: 0 to initialize printer (1 normal)

C_1: LF and CR (0 no auto LF)

C_0: 0 normal but if 1

(set it to output data to printer)

* Out - instruction

Out Port, register

* In - instructio

In register, Port

* Register must be AL
* Port must be number in Dx

Scanned with CamScanner

Ex: initialize the Data port to 00H

```
Mov AL, 00H
Mov Dx, 378H
Out Dx, AL
```

Ex: Send 17H to The Data port

```
Mov AL, 17H
Mov Dx, 378H
Out Dx, AL
```

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	0	0	1	0	1	1	1

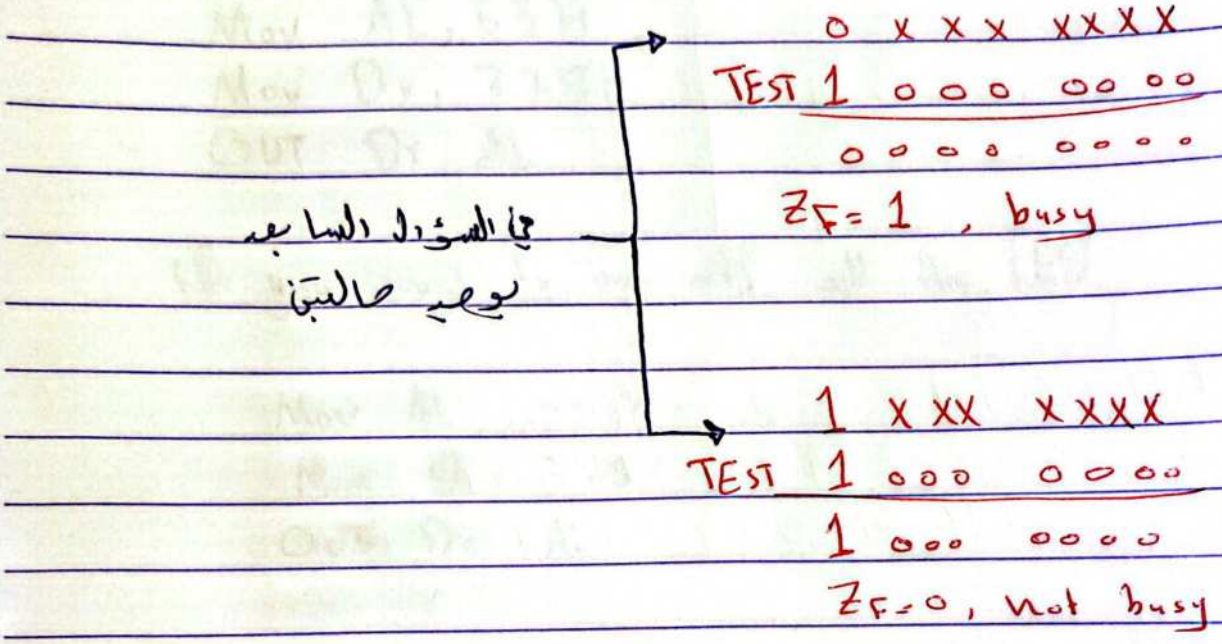
Ex: Address of Status port is 379H

```
Mov Dx, 379H
In  AL, Dx
```

Ex: Test this value to decide the status of printer
Test value 80H = 10000000B if ZF set
the printer is ~~bye~~ busy

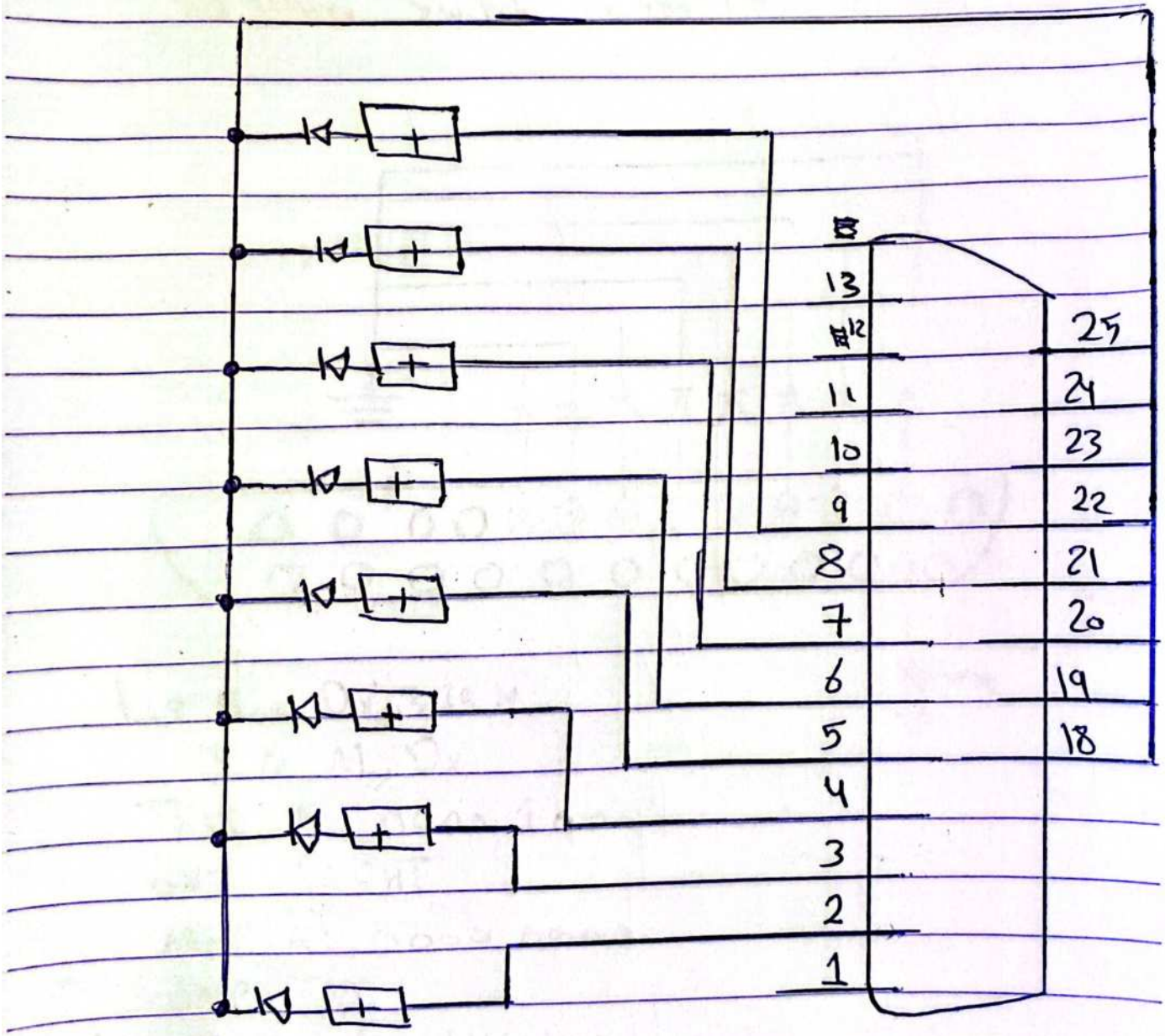
```
    Mov Dx, 379H
busy: IN  AL, Dx
TEST  AL, 80H
  JZ  busy
```

Scanned with CamScanner



* Ex: there is an easy example of Interfacing LEDs using the 8 Data Port in SPP mode. The resistor are there to limit the current on the LEDs.

Scanned with CamScanner



Scanned with CamScanner

if you want to turn LED D_1, D_3, D_5, D_7

```
Mov AL, 55H
Mov Dx, 378H
OUT Dx, AL
```

if you want to turn off all the LED

```
Mov AL, 00H
Mov Dx, 378H
Out Dx, AL
```

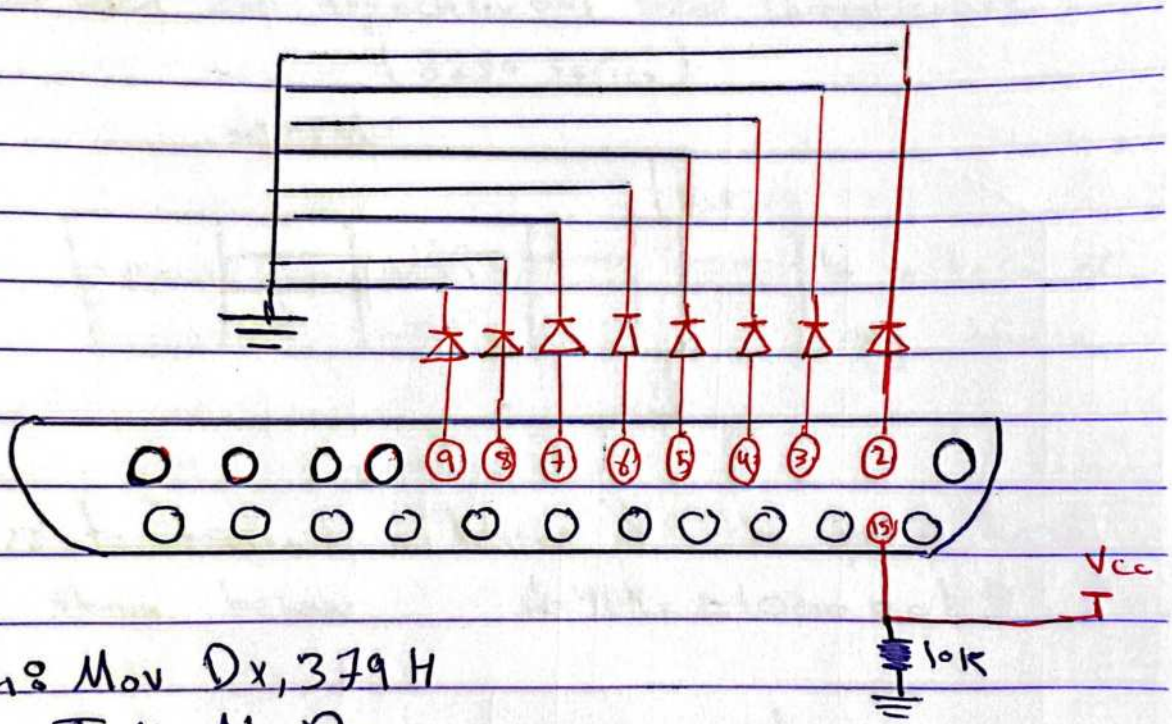
ZerOne ^_^

team

Scanned with CamScanner

Ex: Connect 8 LEDs as in part 1 and connect a switch as shown in the following figure.

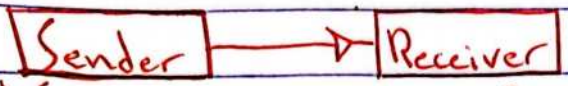
1. Write switch is On LEDs Light ON
2. When Switch is off LEDs Lights off




```
L1: Mov Dx, 379H
    IN AL, Dx
    TEST AL, 0000 1000
    JNZ LIGHT
    Mov AL, 0000 0000
    JMP END
LIGHT: Mov AL, 1111 1111
END1: Mov Dx, 378H
    OUT Dx, AL
END: JMP L1
```

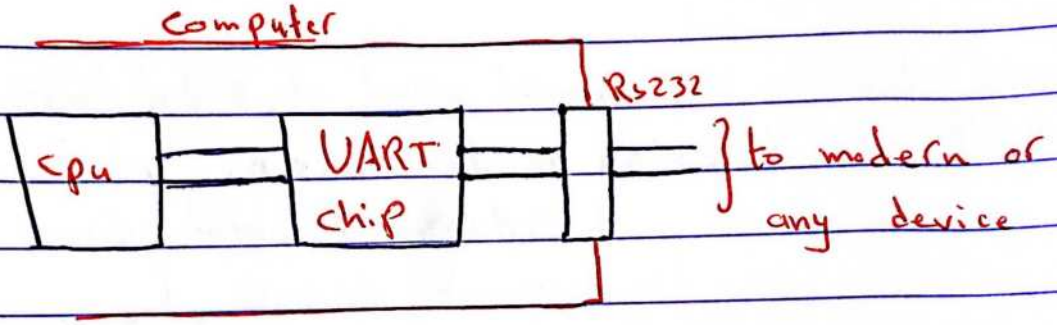
Scanned with CamScanner

Serial Port

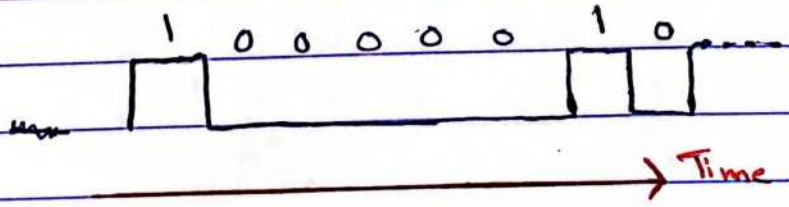


"Serial ~~Port~~ Data Transfer"

UART chip (Universal Asynchronous Receiver Transmitter)
⇒ used for asynchronous serial communication
(8250 series)

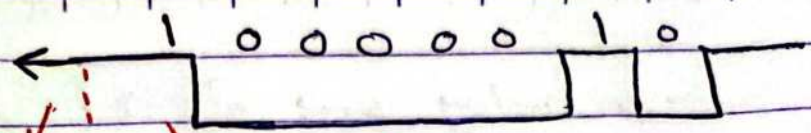


Ex: to transmit a letter 'A' the pulse train shown below A = 41H = 01000001 B



Note: While no transmission is taking place, the signal on the wire between a computer and a Serial device is kept at high

Scanned with CamScanner



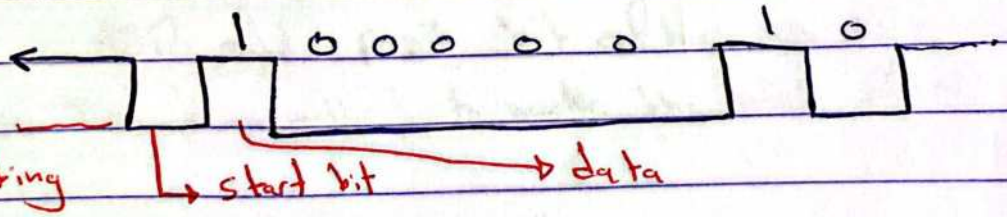
waiting for transmition to begin

this 1 gate lost because the device dose not know the Transmission has begin until the next 0

* Transmit / Receive protocol

→ How the Data is packed

☐ start bit : to begin transmission without missing any data, a start bit is added to the front of each data character



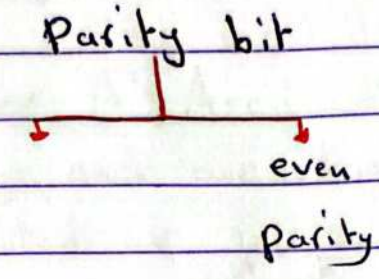
Note: the LSB is sent out first

'A' = 0 1 0 0 0 0 0 1 0

Data

start

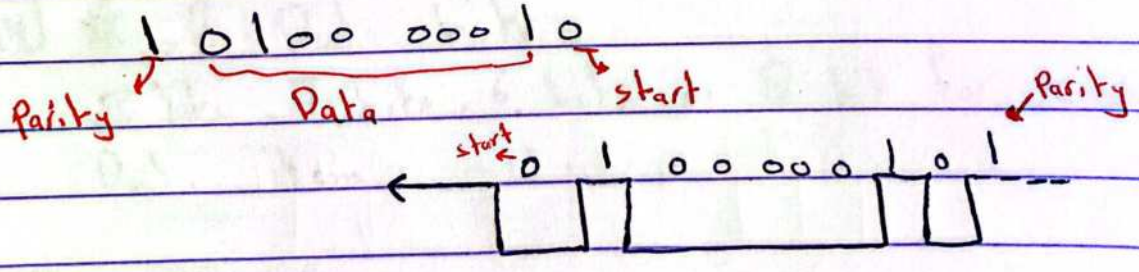
[2] Parity bit :- In some system in order to maintain the Data integrity the parity bit of the charictec byte is included in the data forme



→ Error delection

Ex:- odd parity :-

ADD odd parity bit after the Last data bit to be transmitted to make the total # of is odd

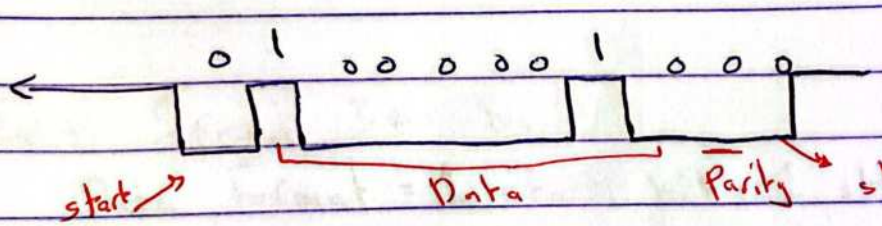


if it was corrupted to 10100 0011 0 then an error would be signaled because 1010000110 cannot be the code of any real data item under our odd parity system

Scanned with CamScanner

[3] stop bits: are added to gave the device a chance to keep up with the speed of the electrical signals
→ just one stop bits is added usually

Ex: the letter 'A', Ascii code 01000001 with even parity and one stop bit is actually transmitted as 0 0 01000001 0
stop parity data star



[4] # of Data bits
8 bits, 7 bits, 6 bits or 5 bit for each Data item

[5] Speed of transmission
→ The basic unit is band (bit per seconed) (bps)

Scanned with CamScanner

Ex: Assume

① Data format = 1 start bit, 7 Data bits even parity and 2 stop bits

→ That is, 11 bits in all

② the teletype device, Print 10 characters per second

what is the Transmission speed?

Speed = 10 * 11 = 110 baud (110 bps)

Ex: Speed 300 baud

Data Format = 1 start bit, 1 stop bit 7 Data bits and odd parity.

→ 300 baud was Exactly equivalent to ..... characters per second

300 = 30 character per second

ZerOne ^^

beam

Scanned with CamScanner